

# Implementación de Restricciones

Maapeo entre requerimientos funcionales y su implementación SQL

Jesdreel Daniel Mata Gómez

18 de julio de 2026

## Índice

|                                                                                                             |          |
|-------------------------------------------------------------------------------------------------------------|----------|
| <b>1. Introducción</b>                                                                                      | <b>2</b> |
| <b>2. Resumen de tipos de restricciones utilizadas</b>                                                      | <b>2</b> |
| <b>3. Maapeo detallado</b>                                                                                  | <b>3</b> |
| RF-01 — La edad mínima del socio es de 19 años cumplidos . . . . .                                          | 3        |
| RF-02 — El aporte inicial mínimo para abrir cuenta es de \$3,000.00 . . . . .                               | 3        |
| RF-03 — Un socio sólo puede tener una cuenta de ahorro . . . . .                                            | 3        |
| RF-04 — Los depósitos deben estar entre \$100 y \$10,000 . . . . .                                          | 3        |
| RF-05 — Interés fijo de 20% mensual para cuentas de ahorro . . . . .                                        | 4        |
| RF-06 — Interés fijo de 5% mensual para préstamos . . . . .                                                 | 4        |
| RF-07 — Un préstamo no puede exceder \$50,000 . . . . .                                                     | 4        |
| RF-08 — Un préstamo no puede exceder 2 veces el saldo de la cuenta . . . . .                                | 4        |
| RF-09 — Máximo 2 préstamos activos por socio . . . . .                                                      | 5        |
| RF-10 — Penalización fija del 40% por atraso . . . . .                                                      | 5        |
| RF-11 — Método de pago American Express prohibido . . . . .                                                 | 5        |
| RF-12 — Beneficiarios: de 1 a 4, sumando exactamente 100% . . . . .                                         | 5        |
| RF-13 — La defunción de un socio salda automáticamente sus préstamos . . . . .                              | 6        |
| RF-14 — CURP con exactamente 18 caracteres, RFC con 12 o 13 . . . . .                                       | 6        |
| RF-15 — Cada socio con documentación única (sin repetir tipo) . . . . .                                     | 7        |
| RF-16 — El saldo de las cuentas nunca puede ser negativo . . . . .                                          | 7        |
| RF-17 — El monto de cualquier transacción debe ser positivo . . . . .                                       | 7        |
| RF-18 — Los estados de los objetos deben ser valores conocidos . . . . .                                    | 7        |
| RF-19 — Un solo abono mensual por préstamo . . . . .                                                        | 7        |
| RF-20 — Capital y presupuesto no pueden ser negativos, y el ejercido no puede exceder al asignado . . . . . | 7        |
| <b>4. Resumen visual: requisitos vs. implementación</b>                                                     | <b>8</b> |
| <b>5. Blindaje ante inserciones aleatorias</b>                                                              | <b>8</b> |

## 1. Introducción

Este documento correlaciona cada **requerimiento funcional** del sistema Caja Popular con la **instrucción SQL específica** que lo implementa en la base de datos BD\_JesdreeL\_Daniel\_Mata\_Gomez. Para cada restricción se indica el tipo (CHECK, UNIQUE, trigger, función), la ubicación dentro del script y una breve descripción del comportamiento.

## 2. Resumen de tipos de restricciones utilizadas

| Tipo        | Cantidad | Ejemplos                                |
|-------------|----------|-----------------------------------------|
| PRIMARY KEY | 11       | Una por tabla                           |
| FOREIGN KEY | 13       | Relaciones entre tablas                 |
| UNIQUE      | 6        | curp, rfc, email, ejercicio_anio, ...   |
| NOT NULL    | +30      | Columnas obligatorias                   |
| CHECK       | 19       | Estados, montos, dominios               |
| DEFAULT     | +15      | Fechas actuales, saldos iniciales, etc. |
| Triggers    | 5        | Reglas complejas                        |
| Funciones   | 5        | Lógica reutilizable                     |

### 3. Mapeo detallado

#### RF-01 — La edad mínima del socio es de 19 años cumplidos

**Implementación:** Trigger `trg_socio_reglas` sobre la tabla `socios`, activa la función `check_socio_reglas()`.

```
CREATE OR REPLACE FUNCTION check_socio_reglas() RETURNS TRIGGER AS $$
BEGIN
    IF NEW.fecha_nacimiento > CURRENT_DATE - INTERVAL '19 years' THEN
        RAISE EXCEPTION 'El socio debe tener 19 años o mas cumplidos';
    END IF;
    ...
```

**Descripción:** Antes de cada INSERT o UPDATE en `socios`, la función calcula la edad a partir de `fecha_nacimiento` y `CURRENT_DATE`. Si el socio no ha cumplido los 19 años, la operación es rechazada con un mensaje claro. Se implementa como trigger porque los CHECK constraints no pueden usar `CURRENT_DATE` de forma segura.

---

#### RF-02 — El aporte inicial mínimo para abrir cuenta es de \$3,000.00

**Implementación:** doble seguro con CHECK en `socios.aporte_inicial` y trigger `trg_socio_reglas`.

```
CONSTRAINT chk_socio_aporte CHECK (aporte_inicial >= 3000.00)

IF NEW.aporte_inicial < 3000.00 THEN
    RAISE EXCEPTION 'El aporte inicial minimo para abrir cuenta es de 3000.00';
END IF;
```

**Descripción:** el CHECK bloquea cualquier valor inferior a 3000. El trigger duplica la validación con un mensaje más claro para el usuario final.

---

#### RF-03 — Un socio sólo puede tener una cuenta de ahorro

**Implementación:** restricción UNIQUE sobre `cuentas_ahorro.socio_id`.

```
socio_id INTEGER NOT NULL UNIQUE REFERENCES socios(id)
```

**Descripción:** al intentar insertar una segunda cuenta con el mismo `socio_id`, PostgreSQL lanza `duplicate key value violates unique constraint`.

---

#### RF-04 — Los depósitos deben estar entre \$100 y \$10,000

**Implementación:** CHECK compuesto sobre `transacciones`.

```
CONSTRAINT chk_transaccion_deposito_min_max CHECK (
    tipo <> 'deposito' OR (monto >= 100.00 AND monto <= 10000.00)
)
```

**Descripción:** el CHECK sólo aplica cuando el tipo es `deposito`. Otros tipos (`retiro`, `abono`, `desembolso`) no están sujetos a este rango.

---

## RF-05 — Interés fijo de 20% mensual para cuentas de ahorro

**Implementación:** DEFAULT + CHECK en cuentas\_ahorro.tasa\_interes\_mensual.

```
tasa_interes_mensual NUMERIC(5,4) NOT NULL DEFAULT 0.2000,  
CONSTRAINT chk_tasa_ahorro CHECK (tasa_interes_mensual = 0.2000)
```

**Descripción:** el CHECK impide alterar la tasa aunque se intente por INSERT explícito o UPDATE. Toda cuenta nace con tasa 0.2000 (20%).

---

## RF-06 — Interés fijo de 5% mensual para préstamos

**Implementación:** DEFAULT + CHECK en prestamos.tasa\_interes.

```
tasa_interes NUMERIC(5,4) NOT NULL DEFAULT 0.0500,  
CONSTRAINT chk_prestamo_tasa CHECK (tasa_interes = 0.0500)
```

**Descripción:** todo préstamo tiene tasa 0.0500 (5%). El CHECK bloquea cualquier intento de alterarla.

---

## RF-07 — Un préstamo no puede exceder \$50,000

**Implementación:** CHECK sobre prestamos.monto\_original.

```
CONSTRAINT chk_prestamo_monto_max CHECK (monto_original <= 50000.00)
```

**Descripción:** monto tope absoluto por préstamo. Se aplica en INSERT y UPDATE.

---

## RF-08 — Un préstamo no puede exceder 2 veces el saldo de la cuenta

**Implementación:** Trigger trg\_prestamo\_vs\_saldo con la función check\_prestamo\_vs\_saldo().

```
CREATE OR REPLACE FUNCTION check_prestamo_vs_saldo() RETURNS TRIGGER AS $$  
DECLARE v_saldo NUMERIC(12,2);  
BEGIN  
    SELECT saldo INTO v_saldo FROM cuentas_ahorro WHERE socio_id = NEW.socio_id;  
    IF v_saldo IS NULL THEN  
        RAISE EXCEPTION 'El socio no tiene cuenta de ahorro registrada';  
    END IF;  
    IF NEW.monto_original > v_saldo * 2 THEN  
        RAISE EXCEPTION 'El monto solicitado (%) excede 2 veces el saldo de la cuenta (%)',  
            NEW.monto_original, v_saldo;  
    END IF;  
    ...  
END
```

**Descripción:** antes de insertar un préstamo, la función consulta el saldo actual de la cuenta del socio y verifica la regla.

---

## RF-09 — Máximo 2 préstamos activos por socio

**Implementación:** Trigger trg\_max\_prestamos\_activos con la función check\_max\_prestamos\_activos().

```
CREATE OR REPLACE FUNCTION check_max_prestamos_activos() RETURNS TRIGGER AS $$
DECLARE v_activos INTEGER;
BEGIN
    IF NEW.estado IN ('activo', 'solicitado') THEN
        SELECT COUNT(*) INTO v_activos FROM prestamos
        WHERE socio_id = NEW.socio_id
        AND estado IN ('activo', 'solicitado')
        AND id <> COALESCE(NEW.id, -1);
        IF v_activos >= 2 THEN
            RAISE EXCEPTION 'El socio ya tiene 2 prestamos activos o solicitados (maximo permitido)';
        END IF;
    END IF;
    ...

```

**Descripción:** cuenta cuántos préstamos en estado activo o solicitado tiene el socio. Si ya hay 2, rechaza el nuevo.

---

## RF-10 — Penalización fija del 40% por atraso

**Implementación:** DEFAULT + CHECK en prestamos.penalizacion\_pct.

```
penalizacion_pct NUMERIC(5,4) NOT NULL DEFAULT 0.4000,
CONSTRAINT chk_penalizacion_pct CHECK (penalizacion_pct = 0.4000)
```

**Descripción:** garantiza que todo préstamo penalice al 40%, sin variación.

---

## RF-11 — Método de pago American Express prohibido

**Implementación:** CHECK sobre transacciones.metodo\_pago.

```
CONSTRAINT chk_transaccion_metodo CHECK (metodo_pago IN (
    'SPEI', 'EFECTIVO', 'DEBITO_VISA', 'DEBITO_MASTERCARD', 'DEBITO_CARNET',
    'CREDITO_VISA', 'CREDITO_MASTERCARD', 'CREDITO_CARNET'
))
```

**Descripción:** el CHECK **no incluye** los valores CREDITO\_AMEX ni DEBITO\_AMEX, por lo que cualquier transacción con esas cadenas es rechazada.

---

## RF-12 — Beneficiarios: de 1 a 4, sumando exactamente 100%

**Implementación:** CHECK sobre porcentaje individual + Trigger trg\_validar\_beneficiarios con validar\_beneficiarios().

```
CONSTRAINT chk_beneficiario_porcentaje CHECK (porcentaje > 0 AND porcentaje <= 100)
```

```

CREATE OR REPLACE FUNCTION validar_beneficiarios() RETURNS TRIGGER AS $$
DECLARE v_count INTEGER; v_suma NUMERIC(6,2); v_socio INTEGER;
BEGIN
    v_socio := COALESCE(NEW.socio_id, OLD.socio_id);
    SELECT COUNT(*), COALESCE(SUM(porcentaje),0) INTO v_count, v_suma
    FROM beneficiarios WHERE socio_id = v_socio;
    IF v_count > 4 THEN
        RAISE EXCEPTION 'Un socio no puede tener mas de 4 beneficiarios';
    END IF;
    IF v_suma > 100.00 THEN
        RAISE EXCEPTION 'La suma de porcentajes de beneficiarios no puede exceder 100 (actual: %)';
    END IF;
    UPDATE socios SET beneficiarios_completos = (v_count BETWEEN 1 AND 4 AND v_suma = 100.00)
    WHERE id = v_socio;
    ...

```

**Descripción:** el trigger actualiza también el campo socios.beneficiarios\_completos a TRUE cuando la suma llega exactamente a 100 con entre 1 y 4 beneficiarios.

---

## RF-13 — La defunción de un socio salda automáticamente sus préstamos

**Implementación:** Trigger trg\_defuncion\_salda\_prestamo con saldar\_prestamo\_por\_defuncion().

```

CREATE OR REPLACE FUNCTION saldar_prestamo_por_defuncion() RETURNS TRIGGER AS $$
BEGIN
    IF NEW.fecha_defuncion IS NOT NULL
    AND (OLD.fecha_defuncion IS NULL OR OLD.fecha_defuncion <> NEW.fecha_defuncion) THEN
        UPDATE prestamos
        SET estado = 'saldado_defuncion',
            saldo_pendiente = 0,
            motivo_cancelacion = 'Defuncion del socio'
        WHERE socio_id = NEW.id
        AND estado IN ('activo', 'solicitado');
        NEW.estado := 'fallecido';
    END IF;
    ...

```

**Descripción:** al registrar fecha\_defuncion, se marcan todos los préstamos vigentes como saldado\_defuncion, se limpia el saldo, se anota el motivo y se pone al socio en estado fallecido.

---

## RF-14 — CURP con exactamente 18 caracteres, RFC con 12 o 13

**Implementación:** CHECK de longitud en socios.

```

CONSTRAINT chk_socio_curp_largo CHECK (char_length(curp) = 18),
CONSTRAINT chk_socio_rfc_largo CHECK (char_length(rfc) BETWEEN 12 AND 13)

```

**Descripción:** rechaza CURP y RFC con longitudes fuera de norma.

---

### RF-15 — Cada socio con documentación única (sin repetir tipo)

**Implementación:** UNIQUE compuesto sobre documentos\_socio (socio\_id, tipo).

**CONSTRAINT** uq\_documento\_por\_socio **UNIQUE** (socio\_id, tipo)

---

**Descripción:** un mismo socio no puede registrar dos veces el mismo tipo de documento (por ejemplo dos INE).

### RF-16 — El saldo de las cuentas nunca puede ser negativo

**Implementación:** CHECK sobre cuentas\_ahorro.saldo.

**CONSTRAINT** chk\_saldo\_no\_negativo **CHECK** (saldo >= 0)

---

### RF-17 — El monto de cualquier transacción debe ser positivo

**Implementación:** CHECK sobre transacciones.monto.

**CONSTRAINT** chk\_transaccion\_monto **CHECK** (monto > 0)

---

### RF-18 — Los estados de los objetos deben ser valores conocidos

**Implementación:** múltiples CHECK IN (...).

**CONSTRAINT** chk\_socio\_estado **CHECK** (estado IN ('activo', 'inactivo', 'suspendido', 'fallecido'))  
**CONSTRAINT** chk\_cuenta\_estado **CHECK** (estado IN ('activa', 'suspendida', 'cerrada'))  
**CONSTRAINT** chk\_prestamo\_estado **CHECK** (estado IN ('solicitado', 'activo', 'pagado', 'cancelado', 'saldo'))  
**CONSTRAINT** chk\_empleado\_rol **CHECK** (rol IN ('admin', 'cajero', 'gerente'))

**Descripción:** los CHECK permiten sólo los valores enumerados. Cualquier otra cadena es rechazada.

---

### RF-19 — Un solo abono mensual por préstamo

**Implementación:** UNIQUE compuesto sobre abonos\_prestamo (prestamo\_id, mes\_correspondiente).

**CONSTRAINT** uq\_abono\_prestamo\_mes **UNIQUE** (prestamo\_id, mes\_correspondiente)

---

### RF-20 — Capital y presupuesto no pueden ser negativos, y el ejercido no puede exceder al asignado

**Implementación:** CHECKs sobre capital\_caja y presupuesto\_operativo.

**CONSTRAINT** chk\_capital\_no\_negativo **CHECK** (monto >= 0)  
**CONSTRAINT** chk\_presupuesto\_positivo **CHECK** (monto\_asignado > 0)  
**CONSTRAINT** chk\_presupuesto\_ejercido **CHECK** (monto\_ejercido >= 0 AND monto\_ejercido <= monto\_asignado)

## 4. Resumen visual: requisitos vs. implementación

| Requisito                           | Tipo de implementación    | Objeto                                       |
|-------------------------------------|---------------------------|----------------------------------------------|
| RF-01 Edad mínima 19                | Trigger + función         | trg_socio_reglas                             |
| RF-02 Aporte >= 3000                | CHECK + trigger           | chk_socio_aporte                             |
| RF-03 Una cuenta por socio          | UNIQUE                    | cuentas_ahorro.socio_id                      |
| RF-04 Depósito 100-10000            | CHECK                     | chk_transaccion_deposito_min_max             |
| RF-05 Interés ahorro 20%            | DEFAULT + CHECK           | chk_tasa_ahorro                              |
| RF-06 Interés préstamo 5%           | DEFAULT + CHECK           | chk_prestamo_tasa                            |
| RF-07 Préstamo <= 50000             | CHECK                     | chk_prestamo_monto_max                       |
| RF-08 Préstamo <= 2 x saldo         | Trigger + función         | trg_prestamo_vs_saldo                        |
| RF-09 Máx 2 préstamos activos       | Trigger + función         | trg_max_prestamos_activos                    |
| RF-10 Penalización 40%              | DEFAULT + CHECK           | chk_penalizacion_pct                         |
| RF-11 AMEX prohibido                | CHECK                     | chk_transaccion_metodo                       |
| RF-12 Beneficiarios 1-4 sumando 100 | CHECK + trigger + función | trg_validar_beneficiarios                    |
| RF-13 Defunción salda préstamos     | Trigger + función         | trg_defuncion_salda_prestamo                 |
| RF-14 Longitud CURP/RFC             | CHECK                     | chk_socio_curp_largo,<br>chk_socio_rfc_largo |
| RF-15 Documentos únicos por socio   | UNIQUE compuesto          | uq_documento_por_socio                       |
| RF-16 Saldo no negativo             | CHECK                     | chk_saldo_no_negativo                        |
| RF-17 Monto de transacción positivo | CHECK                     | chk_transaccion_monto                        |
| RF-18 Estados válidos               | CHECK IN (...)            | varios                                       |
| RF-19 Un abono por mes por préstamo | UNIQUE compuesto          | uq_abono_prestamo_mes                        |
| RF-20 Presupuesto coherente         | CHECK                     | chk_presupuesto_ejercido                     |

## 5. Blindaje ante inserciones aleatorias

Cada regla anterior aplica automáticamente. Ejemplos de intentos que la base rechazará con un mensaje explícito:

- Insertar un socio con `fecha_nacimiento = '2010-01-01'` → error de edad.
- Insertar un socio con `aporte_inicial = 1000` → error de aporte.
- Insertar dos cuentas para el mismo `socio_id` → error de unicidad.
- Registrar una transacción con `metodo_pago = 'CREDITO_AMEX'` → error de CHECK.
- Registrar un depósito de 50 pesos → error de rango.
- Registrar un préstamo de 60000 → error de tope máximo.
- Registrar un préstamo de 30000 para un socio con saldo 8500 → error de 2x saldo.
- Registrar el tercer préstamo activo del mismo socio → error de máximo.
- Registrar beneficiarios cuya suma supere 100 → error del trigger.
- Marcar `fecha_defuncion` de un socio → sus préstamos activos se marcan `saldado_defuncion` automáticamente.